REAL-TIME MARKETPLACE INTELLIGENCE PLATFORM

End-to-End Data Engineering Portfolio Project Blueprint

Target profile: Data Engineer / Big Data Engineer / GCP Data Engineer

Core stack: Python • PostgreSQL • Debezium • Kafka • PySpark • GCS • BigQuery • Airflow • Docker

Core learning themes: Batch + Streaming • CDC • Data Modeling • Data Skew • Spark Optimization • Data Quality • Observability • Cost Optimization

The objective is not to build a toy Kafka pipeline. The objective is to build a small, production-style marketplace data platform and deliberately solve real data-engineering problems.

1. Project Overview

This project simulates the data platform of a large online marketplace. Customers search and view products, sellers update inventory and listings, orders are created, payments are processed, and operational systems continuously change. The platform must ingest these changes, process both streaming and batch workloads, model the data for analytics, validate data quality, monitor freshness and pipeline health, and keep cloud/query costs under control.

The project is designed specifically for a Data Engineering interview. Every technology has a reason to exist, and each phase creates an engineering problem that can be measured and explained.

What the finished system demonstrates

- Ingestion: PostgreSQL transactional data plus event streams.
- CDC: Debezium captures INSERT/UPDATE/DELETE changes from PostgreSQL.
- Streaming: Kafka transports events and Spark Structured Streaming processes them.
- Batch: PySpark handles historical and daily workloads.
- Warehouse: BigQuery stores curated analytical datasets.
- Modeling: Star schema, fact/dimension tables and SCD Type 2 history.
- Orchestration: Airflow manages dependencies, retries, incremental loads and backfills.
- Quality: Schema, null, duplicate, referential-integrity and freshness checks.
- Performance: Data skew, salting, repartitioning, broadcast joins and shuffle analysis.
- Cost: Partitioning, clustering, column pruning and incremental processing.
- Observability: Pipeline duration, rows, failures, freshness and bytes processed.

2. High-Level Architecture



```
Monitoring ■■■ freshness / failures / rows / duration / query cost
```

Architecture principle

Keep the raw layer immutable where practical, make transformations reproducible, make downstream writes idempotent, and make every pipeline measurable. The architecture should be explainable on a whiteboard in five minutes.

3. Business Scope and Use Cases

The simulated marketplace contains customers, sellers, products, orders, payments, inventory, locations and user activity. The analytical platform answers questions such as:

- Which sellers generated the highest GMV today and over the last 30/90 days?
- Which products are trending based on views, orders and conversion rate?
- Which sellers have inventory risk?
- What is the order-to-payment success rate?
- How many active users and sellers were observed per hour/day?
- Is a pipeline delayed or producing abnormal row counts?
- Which queries consume the most BigQuery bytes?

Non-goals

- Do not spend most of the project building a polished e-commerce frontend.
- Do not add technologies merely for resume keywords.
- Do not claim production-scale volume if the experiment was run at smaller scale.
- Do not optimize blindly; capture a baseline, change one thing, then benchmark again.

4. Dataset Design

Start small enough to run locally, but make the generator scalable. The generator should accept row counts, date ranges and skew parameters from the command line.

Entity	Suggested initial size	Purpose
customers	100K–1M	Customer dimension
sellers	10K–100K	Seller dimension; skew driver
products	100K–1M	Product catalog
orders	1M–10M	Primary transaction fact
payments	1M–10M	Payment lifecycle
inventory_events	5M+	Inventory changes
product_views	10M+	High-volume event stream
search_events	10M+	User behavior

Make the generator deterministic where possible using a seed. This makes performance comparisons reproducible.

5. Data Model

The warehouse should use a star-schema style design. Define the grain of every fact table before writing transformations.

```
DIMENSIONS
-----------
dim_customer(customer_sk, customer_id, name, city, state, valid_from, valid_to, is_current)
dim_seller(seller_sk, seller_id, name, category, city, state, valid_from, valid_to, is_current)
dim_product(product_sk, product_id, seller_id, category, price, valid_from, valid_to, is_current)
dim_date(date_sk, date, day, week, month, quarter, year)

FACTS
-----
fact_orders(order_id, customer_sk, seller_sk, product_sk, date_sk,
order_ts, quantity, order_amount, status)

fact_payments(payment_id, order_id, date_sk, payment_ts,
payment_amount, payment_status)

fact_product_views(view_id, customer_sk, product_sk, seller_sk,
event_ts, device, session_id)

fact_inventory(inventory_event_id, product_sk, seller_sk,
event_ts, quantity_delta, event_type)
```

Grain examples

- fact_orders: one row per order line (or explicitly one row per order; choose one and document it).
- fact_payments: one row per payment attempt/event.
- fact_product_views: one row per product-view event.
- fact_inventory: one row per inventory change event.

6. Slowly Changing Dimensions — SCD Type 2

For attributes where historical correctness matters, retain versions instead of overwriting the previous record.

```
seller_id | city | valid_from | valid_to | is_current
--------- | ------ | ---------- | ---------- | ----------
101 | Delhi | 2026-01-01 | 2026-06-10 | false
101 | Mumbai | 2026-06-11 | 9999-12-31 | true
```

Implementation approach: match on business key, detect changed attributes, expire the previous version, then insert the new version. Make the process idempotent so rerunning the same partition does not create duplicate history.

7. OLTP and Data Generation

PostgreSQL acts as the operational source. Create normalized transactional tables and populate them through the Python generator. Keep transaction-like updates separate from analytical transformations.

```
generator/
config.yaml
customers.py
sellers.py
products.py
orders.py
events.py
main.py

Example:
python generate.py --customers 100000 --sellers 10000 --orders 1000000 --skew true
```

Inject realistic behavior

- A small number of sellers should account for a disproportionate share of orders.
- Popular products should receive more views.
- Orders should have realistic status transitions.
- Payment failures should occur at a configurable rate.
- Inventory should decrease on successful order events.
- Late events should be generated occasionally to test event-time processing.

8. CDC with Debezium

Configure PostgreSQL logical replication/WAL-based CDC and Debezium to capture changes. The CDC stream should preserve the operation type and source metadata.

```
PostgreSQL
■
■ WAL / logical replication
▼
Debezium
■
■ CDC event
▼
Kafka topic: marketplace.orders
■
■■■ INSERT
■■■ UPDATE
■■■ DELETE
```

Important interview concepts

- Why CDC instead of repeatedly extracting the full source table?
- How are deletes represented?
- What is an offset and how is recovery handled?
- How do you make the sink idempotent?
- What happens if the consumer crashes after processing but before committing its offset?
- How do you handle schema evolution?

9. Kafka Design

Create separate topics by event domain. Start with a small number of partitions locally and scale partitions only when you have a reason.

Topic	Key	Typical events
orders	order_id	created, updated, cancelled
payments	order_id/payment_id	initiated, success, failed
inventory	product_id	stock_added, stock_reserved, stock_released
product_events	product_id	viewed, clicked, searched
user_events	customer_id/session_id	search, login, session events

Partition-key decision

Choose keys based on the ordering and distribution requirement. A key that preserves per-entity ordering can also create skew if a few entities dominate traffic. This is a good place to demonstrate the trade-off between ordering and load distribution.

10. Spark Structured Streaming

```
Kafka
■
▼
readStream
■
■■■ parse JSON / schema
■■■ validate
■■■ deduplicate
■■■ watermark
■■■ aggregate
■■■ write sink
■
▼
BigQuery / curated storage
```

Use event time rather than processing time when calculating business windows. Add watermarks to bound state and deliberately inject late events to demonstrate why watermarks matter.

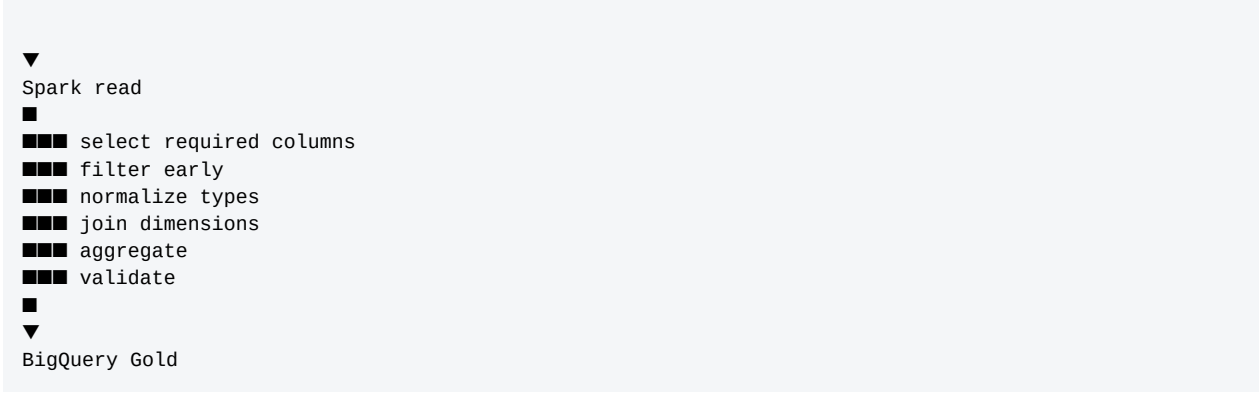
Streaming scenarios to implement

- 5-minute order count by seller.
- 15-minute GMV by category.
- Active products based on recent views.
- Inventory alerts when stock crosses a threshold.
- Deduplication of repeated events using a stable event identifier.
- Late-event handling using event-time windows and watermarks.

11. Batch Processing with PySpark

Use batch Spark for historical loads, daily aggregates and replay/reprocessing. Read raw data from GCS or another durable raw layer, transform it, and publish curated tables to BigQuery.

```
Raw files
■
```



Keep transformations modular. A useful pattern is: extract → standardize → enrich → aggregate → quality-check → publish.

12. Data Skew Lab — Core Performance Experiment

This is one of the most important parts of the project. Do not simply say 'I handled skew'. Create a controlled experiment where skew is measurable.

Experiment design

```
Case A — Uniform distribution
seller_id is approximately evenly distributed.

Case B — Skewed distribution
top 1% of sellers generate ~60% of orders.

Join:
orders JOIN sellers ON orders.seller_id = sellers.seller_id
```

Observe Spark UI metrics: task duration, input size per task, shuffle read/write, spill and the maximum-to-median task duration ratio.

Optimization ladder

- Baseline: regular join; record the slowest task and total runtime.
- Repartition: test whether better partitioning improves distribution.
- Broadcast: broadcast only when the dimension side is genuinely small enough.
- Salting: split heavy keys into salted sub-keys and replicate the matching dimension rows as needed.
- Compare: runtime, shuffle volume, memory pressure and correctness.

```
Example salting concept:

orders:
seller_id=101, salt=0
seller_id=101, salt=1
seller_id=101, salt=2
...

seller dimension:
seller_id=101 replicated for salts 0..N

Join on (seller_id, salt)
```

Do not assume salting is always better. It adds complexity and can increase data volume. The interview-worthy answer is: first detect skew, then choose the least expensive technique that fixes the bottleneck.

13. Spark Join Strategy Lab

Technique	When useful	Main risk
Broadcast hash join	One side is small	Driver/executor memory pressure
Shuffle hash/sort-merge	Large distributed joins	Shuffle cost
Repartition	Poor partition distribution	Extra shuffle
Salting	Few hot keys dominate	Data expansion and complexity
Pre-aggregation	Many duplicate keys/events	Must preserve business semantics

14. BigQuery Data Warehouse Layer

Use separate logical layers/datasets where practical. A simple layout is raw/bronze, silver and gold. Gold tables should be business-facing and stable.

```
GCP Project
■
■■■ raw
■ ■■■ marketplace_events
■
■■■ silver
■ ■■■ orders_clean
■ ■■■ payments_clean
■ ■■■ inventory_clean
■
■■■ gold
■■■ seller_daily_metrics
■■■ product_daily_metrics
■■■ customer_daily_metrics
■■■ marketplace_daily_metrics
```

Partitioning and clustering

• Partition large fact tables by the date column most frequently used for pruning.

• Cluster on frequently filtered/joined dimensions such as seller_id, product_id or category when justified.

• Avoid wrapping partition columns in functions when it prevents effective pruning.

• Select only required columns instead of SELECT *.

• Prefer incremental transformations over rebuilding entire historical tables when the workload permits.

15. BigQuery Cost Optimization Experiment

Create a baseline query, record bytes processed and runtime, then optimize it. Keep screenshots or query-job metadata in the repository.

```
Baseline
SELECT *
FROM orders
WHERE DATE(order_ts) = CURRENT_DATE();

Optimized pattern
SELECT order_id, seller_id, order_amount
FROM orders
WHERE order_date = CURRENT_DATE();
```

The exact improvement must be measured in your environment. Never put an invented percentage on the resume.

Useful metrics

• bytes processed

• bytes billed

• slot/compute usage where available

• wall-clock duration

• rows returned

• partition/cluster pruning effectiveness

16. Airflow Orchestration

```
daily_marketplace_pipeline
■
■■■ wait_for_source
■
■■■ ingest_raw
■
■■■ validate_schema
■
■■■ transform_dimensions
■
■■■ transform_facts
■
■■■ run_quality_checks
■
■■■ publish_gold
■
■■■ publish_metrics
```

Implement retries, sensible timeouts, task-level logging, parameterized dates and idempotent writes. Add a backfill mode so a historical date can be rerun without corrupting current data.

17. Data Quality Framework

Build a lightweight reusable Python/PySpark quality layer rather than scattering ad-hoc checks throughout every job.

Check	Example
Not-null	order_id, seller_id, event_ts
Uniqueness	order_id / event_id
Range	quantity > 0, amount >= 0
Referential integrity	seller_id exists in seller dimension
Freshness	latest event within expected SLA
Volume	row count within expected bounds
Schema	required columns and data types
Distribution	null/duplicate rate does not spike

Quality result table

```
quality_run_id | dataset | check_name | status | observed | threshold | run_ts
-------------- | ------- | ---------- | ------ | -------- | --------- | ------
abc123 | orders | null_check | PASS | 0.01% | <0.10% | ...
```

Failed checks should be queryable and should feed alerts. A quality framework becomes much more useful when it produces structured evidence rather than only log messages.

18. Observability and Monitoring

Track both pipeline health and data health.

- Pipeline duration and task duration.
- Records read/written.
- Failed and rejected records.
- Latest event timestamp and data freshness.
- Kafka consumer lag where measurable.
- BigQuery bytes processed/billed for important jobs.
- Quality-check pass/fail counts.
- Unexpected volume/distribution changes.

Example alert

```
ALERT: seller_daily_metrics
Expected freshness: < 30 minutes
Observed freshness: 74 minutes
Last successful run: 08:11
Current status: FAILED
Likely cause: upstream orders_clean delayed
```

19. Reliability and Idempotency

This is a key interview topic. A pipeline is not production-ready merely because it works once.

Design questions to answer

- What happens if the job runs twice?
- What happens if Spark retries a task?
- What happens if Kafka delivers an event again?
- What happens if an Airflow task fails after partially writing data?
- Can a date partition be safely rebuilt?
- How are deletes propagated?
- How do you recover from a bad deployment?

Recommended strategy

Use stable business/event identifiers, deduplicate at the appropriate layer, write by deterministic partitions or MERGE keys, and make reruns produce the same final state. Separate temporary/staging outputs from published tables where useful.

20. Testing Strategy

Layer	Tests
Generator	row counts, uniqueness, distribution/skew
CDC	insert/update/delete propagation
Spark	transformation correctness, joins, null handling
Streaming	deduplication, late events, watermark behavior
Warehouse	schema, partition presence, business aggregates
Data quality	PASS/FAIL behavior and thresholds
Airflow	DAG parse, dependencies, retries, parameter handling
End-to-end	source event → curated table

Create a small deterministic fixture dataset for unit tests. Large data should be reserved for integration/performance tests.

21. Failure Scenarios to Demonstrate

A strong project deliberately tests failure instead of only showing the happy path.

- Kafka consumer restarts.
- Duplicate event arrives.
- Out-of-order event arrives.
- Late event arrives beyond watermark.
- Airflow transformation task fails and retries.
- BigQuery write fails.
- Source schema gains a new field.
- A required column becomes null.
- Input volume suddenly doubles.
- One seller becomes a hot key.
- A partition is accidentally processed twice.

For each failure, document detection → impact → recovery → prevention.

22. Repository Structure

```
marketplace-data-platform/
■
■■■ README.md
■■■ docker-compose.yml
■■■ .env.example
■
■■■ data_generator/
■ ■■■ config.yaml
■ ■■■ customers.py
■ ■■■ sellers.py
■ ■■■ products.py
■ ■■■ orders.py
■ ■■■ main.py
■
■■■ schemas/
■ ■■■ kafka/
■ ■■■ warehouse/
■
■■■ spark/
■ ■■■ batch/
■ ■■■ streaming/
■ ■■■ jobs/
■
■■■ airflow/
■ ■■■ dags/
■ ■■■ plugins/
■
■■■ quality/
■ ■■■ checks.py
■ ■■■ rules.yaml
■
■■■ sql/
■ ■■■ bronze/
■ ■■■ silver/
■ ■■■ gold/
■
■■■ monitoring/
■ ■■■ metrics/
■
■■■ tests/
```

```
■ ■■■ unit/
■ ■■■ integration/
■
■■■ docs/
■■■ architecture.md
■■■ data-model.md
■■■ skew-experiment.md
■■■ cost-optimization.md
■■■ failure-scenarios.md
```

23. Development Roadmap

Phase	Deliverable	Primary learning
1	Generator + PostgreSQL	OLTP, data generation, distributions
2	Warehouse model	Grain, facts, dimensions, SCD
3	PySpark batch	Transformations, joins, partitions
4	GCS + BigQuery	Cloud storage and warehouse
5	Debezium + Kafka	CDC, partitions, offsets
6	Spark Streaming	Windows, watermarks, state
7	Airflow	Orchestration, retries, backfills
8	Data quality	Validation and SLAs
9	Skew lab	Salting, shuffle, join optimization
10	Cost lab	Partitioning, clustering, pruning
11	Monitoring	Observability and alerting
12	Hardening	Testing, failure recovery, docs

Suggested order

Do not start with Kafka. Start with the data model and a working batch pipeline. Add streaming after you understand the target state. Add optimization after you have a baseline. This keeps the project educational rather than becoming a configuration exercise.

24. Local vs Cloud Setup

Use Docker locally for infrastructure that would otherwise be expensive or operationally heavy. Use GCP selectively for the cloud-specific portions.

Component	Recommended learning setup
PostgreSQL	Docker/local
Kafka	Docker/local
Debezium	Docker/local
Airflow	Docker/local
Spark	Local mode or small managed cluster when needed
GCS	GCP
BigQuery	GCP
Dashboard	Local

Before running large experiments, estimate data volume and query/compute cost. Keep the cloud dataset small until the pipeline is correct.

25. Performance Benchmark Plan

Every optimization should have a before/after record. Keep a CSV or Markdown table in the repository.

Experiment	Baseline	Change	Metrics
Skewed join	Regular join	Salting	runtime, max task, shuffle
Small dimension join	Shuffle	Broadcast	runtime, memory
BigQuery query	Full scan	Partition filter	bytes processed
BigQuery columns	SELECT *	Column pruning	bytes processed
Batch load	Full refresh	Incremental	runtime, rows written
Streaming	No dedup	Event-id dedup	duplicate rate

Benchmark discipline

- Run the same workload before and after.
- Use the same input data.
- Change one major variable at a time.
- Record hardware/runtime configuration.
- Measure more than wall-clock time when possible.
- Verify output correctness after optimization.

26. Resume Version

Once the project is complete, do not copy generic claims. Replace placeholders with your actual measurements.

```
REAL-TIME MARKETPLACE INTELLIGENCE PLATFORM
Python, PySpark, Kafka, Debezium, Airflow, BigQuery, GCS, PostgreSQL, Docker

• Built an end-to-end batch and streaming data platform for marketplace orders,
payments, inventory and user events using PySpark, Kafka, Debezium and BigQuery.

• Designed a star-schema warehouse with fact/dimension models and SCD Type 2
processing for historical seller, product and customer attributes.

• Implemented CDC ingestion from PostgreSQL through Debezium and Kafka and
processed event-time streams using Spark Structured Streaming with
deduplication and watermarking.

• Generated controlled data-skew workloads and optimized Spark joins using
repartitioning, broadcast joins and salting, improving [METRIC] from
[BEFORE] to [AFTER].

• Built Airflow workflows with retries, incremental processing, data-quality
checks and backfill support; monitored freshness, volume and pipeline failures.

• Optimized BigQuery workloads using partitioning, clustering and column
pruning, reducing bytes processed by [METRIC] in benchmark workloads.
```

27. Interview Preparation — Questions You Should Be Able to Answer

The project is successful only if you can explain why every design choice exists.

Core question set

- Why Kafka? Why not write directly from the application to BigQuery?
- Why Debezium? What problem does CDC solve?
- How does Debezium capture DELETE operations?
- What happens when a Kafka consumer crashes?
- How do Kafka partitions affect ordering and parallelism?
- What causes Spark data skew?
- How did you prove that skew existed?
- When would you use a broadcast join?
- How does salting work and what are its drawbacks?
- What is a shuffle and why is it expensive?
- What is the difference between repartition and coalesce?
- What is a watermark in Structured Streaming?
- How do you handle late-arriving data?
- How do you make a pipeline idempotent?
- Why use SCD Type 2?
- What is the grain of your fact table?
- Why partition and cluster BigQuery tables?
- How do you reduce BigQuery cost?
- How do you detect a freshness issue?
- How would you backfill 90 days without duplicating data?
- What happens if a DAG fails halfway through?
- How do you handle schema evolution?
- How do you test a streaming pipeline?
- What was the biggest performance bottleneck and how did you find it?
- What would you change if the data volume increased 100x?

28. What Makes This Project Strong

- It is not a CRUD project: the core deliverable is a data platform.
- It contains both batch and streaming: useful for DE interviews.
- It contains real failure modes: duplicates, late data, skew and retries.
- It contains measurable optimization: Spark and BigQuery benchmarks.

• It contains data modeling: facts, dimensions and SCD Type 2.

• It contains operations: Airflow, quality, freshness and monitoring.

• It complements your professional experience: it lets you demonstrate end-to-end ownership and deeper understanding.

29. Definition of Done

Do not call the project complete because the pipeline runs once. Use this checklist.

• Data generator can reproduce a controlled dataset.

• PostgreSQL source is populated and updated.

• Warehouse grain and schema are documented.

• Batch Spark pipeline works end-to-end.

• CDC pipeline captures insert/update/delete.

• Kafka topics and partition strategy are documented.

• Streaming pipeline handles duplicates and late events.

• Gold tables are queryable in BigQuery.

• Airflow orchestrates daily/incremental processing.

• Data quality checks produce structured results.

• Freshness and pipeline metrics are captured.

• At least one skew experiment is benchmarked.

• At least one BigQuery cost optimization is benchmarked.

• Failure scenarios are tested and documented.

• Unit/integration tests exist.

• README contains architecture, setup and benchmark results.

• Resume bullets use only measured, truthful metrics.

30. Final Guidance

Build this as a learning project, not a resume-decoration project. The strongest outcome is not the number of technologies; it is your ability to explain the complete lifecycle: how data is generated, how it changes, how it is captured, how it moves, how Spark processes it, how the warehouse models it, how Airflow orchestrates it, how quality is verified, how failures are recovered, and how performance/cost are measured.

Recommended first milestone

Start with Python generator → PostgreSQL → star-schema design → PySpark batch → BigQuery. Do not touch Kafka or Debezium until this path is working and you understand the data model. Then add CDC and streaming on top of a stable foundation.

Your final GitHub repository should read like a small engineering case study, not a collection of scripts.